



VERIQTA

BUILD • AUTOMATE • OPERATE • SCALE

★ AUTOMATE
EVERYTHING.
ELIMINATE
MANUAL WORK.

SHELL SCRIPTING

PRODUCTION HANDBOOK

VOLUME 1

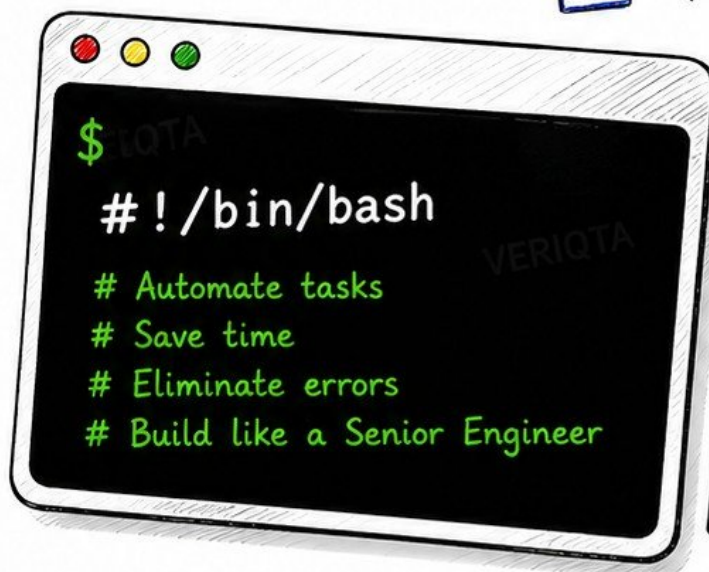
BASH FOUNDATIONS AND PRACTICAL AUTOMATION

- ✓ Bash Fundamentals
- ✓ Automation Techniques
- ✓ Production Best Practices
- ✓ Error Handling & Debugging
- ✓ Real-World Examples
- ✓ Interview-Ready Skills



FROM SIMPLE COMMANDS
TO POWERFUL AUTOMATION

MASTER SHELL. MASTER OPERATIONS.



AUTOMATE
TASKS



SAVE
TIME



REDUCE
ERRORS



WRITE CLEAN
SCRIPTS



IMPROVE
RELIABILITY



SCALE YOUR
IMPACT

ESSENTIAL
FOR DEVOPS,
SRE, CLOUD AND
PLATFORM
ENGINEERS



LEARN IT.
SCRIPT IT.



AUTOMATE IT.
OPERATE IT.



SECURE IT.
SCALE IT.



THINK LIKE A SENIOR.
BUILD LIKE A PRO.

1. INTRODUCTION TO SHELL SCRIPTING

"SHELL SCRIPTING IS THE **GLUE** THAT TURNS COMMANDS INTO **AUTOMATION** AND TASKS INTO **RELIABLE SYSTEMS**."

WHAT IS SHELL SCRIPTING?

Shell scripting is writing a series of commands in a file so the operating system (shell) can execute them sequentially.



It helps you **automate tasks**, manage systems, process text, and build powerful tools.

BASH VS SHELL

SHELL (sh)

- ✓ Generic term for a command-line interpreter
- ✓ Different shells exist (sh, csh, ksh, zsh, bash)
- ✓ Basic features
- ✓ POSIX standard compliant
- ✓ /bin/sh is default on most systems

BASH (bash)

- ✓ Bourne Again Shell
- ✓ Most popular shell on Linux
- ✓ More features and better scripting
- ✓ Advanced features (arrays, functions, [[]], etc.)
- ✓ /bin/bash on most modern systems

VS



Bash is a **specific** shell. All Bash is a shell, but not all shells are **Bash**.

WHY ENGINEERS AUTOMATE TASKS?

- ✓ Save time and reduce repetitive work
- ✓ Improve accuracy and consistency
- ✓ Eliminate human errors
- ✓ Enable large-scale system management
- ✓ Ensure reliability and standardization
- ✓ Focus on solving bigger problems



PRODUCTION USE CASES

- ☁ Server Provisioning & Configuration
- 📁 Backup & Restore Automation
- 📊 Log Rotation & Cleanup
- 🛡 Monitoring & Alerting
- 👥 User & Permission Management
- 🚀 Deployments & Release Automation

SCRIPT EXECUTION WORKFLOW

1. WRITE SCRIPT



Create your script using any editor.

2. SAVE SCRIPT



Save with .sh extension.

3. MAKE EXECUTABLE



chmod +x script.sh

4. EXECUTE SCRIPT



./script.sh

5. GET RESULTS



Output, logs, or changes applied.

SENIOR INSIGHT

Great engineers don't just run commands. They automate, document, validate, and build systems that run without babysitting.

REMEMBER:

A good script today becomes a reliable system tomorrow.



Instagram:
@veriqta



LinkedIn:
VERIQTA



X:
@veriqta



YouTube:
VERIQTA

VERIQTA ❤️

2. CREATING YOUR FIRST SCRIPT

EVERY AUTOMATION JOURNEY BEGINS WITH A SIMPLE SCRIPT.
LET'S BUILD IT THE RIGHT WAY!

① SHEBANG

The shebang (`#!/`) tells the system which interpreter to use to run your script.

COMMON SHEBANGS

```
#!/bin/bash    # Bash (most common)
#!/bin/sh      # POSIX shell
#!/usr/bin/env bash # Portable way (recommended)
```

💡 Always use: `#!/usr/bin/env bash`
It works on more systems.

② FILE EXTENSIONS

Not mandatory, but helps with readability and tooling.

Extension	Purpose
<code>.sh</code>	Shell script (standard)
<code>.bash</code>	Bash-specific script
<code>.ksh</code>	Korn shell script
(none)	Works, but not recommended

★ Use `.sh` for maximum compatibility.

③ SCRIPT PERMISSIONS

Scripts must be **executable**.

```
$ ls -l myscript.sh
-rw-r--r-- 1 user user 120 May 15 10:00 myscript.sh
↓
$ chmod +x myscript.sh
↓
$ ls -l myscript.sh
-rwxr-xr-x 1 user user 120 May 15 10:00 myscript.sh
```

🔒 Without **execute** permission, the system won't run it!

④ RUNNING SCRIPTS

There are several ways to run your script.

	1. Direct Execution (Recommended) <code>\$./myscript.sh</code> Requires executable permission (<code>+x</code>)
	2. Using bash Explicitly <code>\$ bash myscript.sh</code> Does not require <code>+x</code> permission
	3. Using sh (POSIX) <code>\$ sh myscript.sh</code> Runs with system's <code>/bin/sh</code>
	4. Source (For libraries / functions) <code>\$ source myscript.sh</code> Runs in current shell (not a new process)

💡 BEST PRACTICE

- ☑ Use `./script.sh`
- ☑ Keep scripts in version control
- ☑ Test with `bash -n` before running
- ☑ Add error handling from the start

⑤ COMMON STARTUP ERRORS

ERROR	CAUSE	SOLUTION
Permission denied ⊗	Missing execute permission	<code>chmod +x script.sh</code>
bad interpreter: No such file or directory ⊗	Wrong shebang or interpreter not installed	Use <code>#!/usr/bin/env bash</code> or install bash
command not found ⊗	Script run from wrong directory or missing command	Check path, use absolute paths, or install tools
cannot execute binary file ⊗	File has Windows line endings (CRLF)	Convert: <code>dos2unix script.sh</code>
Syntax error: unexpected end of file ⊗	Missing <code>fi</code> , <code>done</code> , or closing quotes	Check Script Structure & Quotes

★ MINIMAL FIRST SCRIPT EXAMPLE

```
1 #!/usr/bin/env bash
2 # My first script
3
4 echo "Hello, VERIQTA!"
4 echo "This is my first automation script."
5
6 exit 0
```

Run:

```
$ chmod +x first.sh
$ ./first.sh
```

Output:

```
Hello, VERIQTA!
This is my first automation script.
$
```



Small scripts today,
powerful systems tomorrow!

"VARIABLES STORE DATA THAT YOUR SCRIPT CAN USE, REUSE AND MANIPULATE. THEY MAKE SCRIPTS DYNAMIC, FLEXIBLE AND POWERFUL."

VERIQTA

1 CREATING VARIABLES

Create a variable by assigning a value.

```
1 name="VERIQTA"
2 age=30
3 city='New York'
4 path=/opt/scripts
5 active=true
```

 No spaces around =
 name=John
 name = John

2 REFERENCING VARIABLES

Use \$ before the variable name to access its value.

```
1 name="VERIQTA"
2 echo "Hello, $name!"           # Hello, VERIQTA!
3 echo "Our name is ${name}."    # Our name is VERIQTA.
```

 \${ } is used when the variable is part of a word or when you want clarity.
Example: \${name}_logs

3 QUOTING

Quoting affects how values are interpreted.

Type	Example	Output / Behavior
No Quotes	msg=Hello World	Word splitting occurs
Double Quotes	msg="Hello World"	Preserves spaces, allows variable expansion
Single Quotes	msg='Hello World'	Preserves spaces, no variable expansion

EXAMPLE

```
1 name="VERIQTA"
2 echo $name           # VERIQTA
3 echo "$name is awesome" # VERIQTA is awesome
4 echo '$name is awesome' # $name is awesome
```

 Use double quotes most of the time in scripts.

4 READ-ONLY VARIABLES

You can prevent a variable from being changed.

```
1 version="1.0.0"
2 readonly version
3 version="2.0.0" # Error: readonly variable
4 echo $version   # 1.0.0
```



Useful for constants like versions, domains, paths, or limits.

5 ENVIRONMENT VARIABLES

Environment variables are available to the current shell and child processes.

```
1 export APP_ENV=production
2 export LOG_LEVEL=info
3 echo $APP_ENV           # production
4 env | grep APP_ENV      # shows in environment list
```



Common built-in variables:

\$HOME \$USER \$PATH \$PWD \$SHELL

Use env or printenv to view all environment variables.

6 PRODUCTION EXAMPLES

USE CASE	EXAMPLE	DESCRIPTION
 Configuration Settings	DB_HOST="db.prod.com" DB_PORT=5432	Store configuration values
 Paths	LOG_DIR="/var/log/myapp" SCRIPT_DIR="\$(pwd)"	Store important paths
 User Input or defaults	USER_INPUT="\${1:-default}"	Use variables with defaults
 Timestamps	TIMESTAMP=\$(date +%F_%T)	Generate dynamic values
 Feature Flags	ENABLE_CACHE=true	Control behavior with flags

BEST PRACTICES

- ☒ Use meaningful, UPPERCASE for global variables.
- ☒ Quote your variables unless you have a strong reason not to.
- ☒ Export variables that must be available to child processes.
- ☒ Use readonly for constants.
- ☒ Avoid overwriting built-in variables.



TIP

Good variable names make your scripts easy to read, maintain and scale.



AVOID

- ☒ Unquoted variables with spaces
- ☒ Hardcoding values repeatedly
- ☒ Using single letters (except loop counters)
- ☒ Storing secrets in plain text



Instagram:
@veriqta



LinkedIn:
VERIQTA



X:
@veriqta



YouTube:
VERIQTA

VERIQTA 

USER INPUT MAKES SCRIPTS DYNAMIC AND INTERACTIVE.
THE `read` COMMAND IS USED TO CAPTURE INPUT FROM USERS.

1 READ COMMAND

The `read` command reads a line of input from the user and stores it in a variable.

```
read variable_name # Basic syntax
```

```
$ read name
$ echo $name VERIQTA
```

NOTE

`read` reads up to a newline (Enter key).

2 PROMPTS

Use `-p` to display a prompt to the user.

```
read -p "Enter your name: " name
echo "Hello, $name!"
```

Example Run:

```
Enter your name: Ann
Hello, Ann!
```

TIP

Always give users clear prompts so they know what to enter.

3 SILENT INPUT (HIDE INPUT)

Use `-s` to hide input (useful for passwords, tokens).

```
read -s -p "Enter your password: " password
echo # Move to new line
echo "Password length: ${#password}"
```



USE CASE

Passwords,
API tokens,
secret keys.

Example Run:

```
Enter your password: *****
Password length: 10
```

4 DEFAULT VALUES

Provide default values if the user enters nothing.

```
read -p "Enter environment [dev]: " env
env=${env:-dev} # dev is default
echo "Environment: $env"
```

★ HOW IT WORKS

If `$env` is empty (user just presses Enter), it uses the default value.

Example Runs:

```
Enter environment [dev]: | Enter environment [dev]: prod
Environment: dev | Environment: prod
(User pressed Enter) | (User entered prod)
```

5 INPUT VALIDATION

Always validate user input to prevent mistakes and ensure correct data.

```
read -p "Enter a number between 1 and 100: " num
while ! [[ $num -- ^[0-9]+$ ]] || [ $num -lt 1 ] || [ $num -gt 100 ]; do
    echo "Invalid input! Enter a number between 1 and 100."
    read -p "Enter a number between 1 and 100: " num
done
echo "You entered: $num"
```

What this does:

- ✓ Checks if input is a number.
- ✓ Checks if number is between 1 and 100.
- ✓ Repeats until valid input is entered.



6 SECURE HANDLING

Handle sensitive input carefully.



Never echo secrets

Avoid printing passwords, tokens or keys.



Clear sensitive variables

Unset variables or overwrite after use.

```
unset password # remove from memory
```



Avoid storing secrets in scripts

Use environment variables or secure vaults.



Limit permissions

Run scripts with least privilege.

💡 BEST PRACTICE

- ✓ Validate all input.
- ✓ Sanitize when needed.
- ✓ Never trust user input.
- ✓ Secure secrets from start to end.



QUICK REFERENCE



Basic Input

`read var`



Prompt

`read -p "Prompt: " var`



Silent Input

`read -s -p "Password: " var`



With Default

`var=${var:-default}`



Validation

Use `while` loops or `regex`



Secure

Don't echo secrets, `unset` when done



Instagram:
@veriqta



LinkedIn:
VERIQTA



X:
@veriqta



YouTube:
VERIQTA

VERIQTA 

5. COMMAND-LINE ARGUMENTS

COMMAND-LINE ARGUMENTS ARE VALUES PASSED TO A SCRIPT WHEN IT IS EXECUTED.
THEY MAKE SCRIPTS FLEXIBLE AND REUSABLE.

① POSITIONAL PARAMETERS

Values passed to the script in order. Accessed with \$1, \$2, \$3 ...



② SPECIAL PARAMETERS OVERVIEW

PARAMETER	MEANING	EXAMPLE
\$0	Name of the script	./deploy.sh
\$1 - \$9	First to ninth arguments	\$1 = dev, \$2 = us-east-1
##	Number of arguments	4
@	All arguments as separate words	"\$1" "\$2" "\$3" "\$4"
*	All arguments as a single word	"dev us-east-1 blue"

\$0 - Script Name

Holds the name/path used to run the script.
echo \$0 # ./deploy.sh

1

\$1 to \$9 - Individual Arguments

Access each argument by its position.
echo \$1 # first argument

#

- Number of Arguments

Returns how many arguments were passed.
echo ## # 4

@

@ - All Arguments (Separate)

Expands to all arguments, preserving spaces.
for arg in "\$@"; do echo \$arg; done

*

* - All Arguments (Single Word)

Expands to all arguments as one word.
echo "\$*" # dev us-east-1 blue

③ EXAMPLE SCRIPT DEMO

```
1 #!/bin/bash
2 # demo.sh
3
4 echo "Script Name : $0"
5 echo "First Arg (1) : $1"
6 echo "Second Arg (2) : $2"
7 echo "Third Arg (3) : $3"
8 echo "Total Args (##) : ##"
9 echo "ALL Args (@) : @"
10 echo "ALL Args (*) : *"
```

Run Example

\$./demo.sh prod ap-south-1 green

Output:

```
Script Name : ./demo.sh
First Arg (1) : prod
Second Arg (2) : ap-south-1
Third Arg (3) : green
Total Args (##) : 3
All Args (@) : prod ap-south-1 green
All Args (*) : prod ap-south-1 green
```

④ PRODUCTION EXAMPLES

USE CASE	EXAMPLE COMMAND	INSIDE SCRIPT (USE)	PURPOSE
Environment Selection	./deploy.sh dev	ENV="\$1"	Deploy to dev/stage/prod
Region / Zone	./backup.sh us-east-1	REGION="\$1"	Pass AWS region
File Input	./process.sh data.csv	FILE="\$1"	Process specific file
User / Group	./create-user.sh john ops	USER="\$1" GROUP="\$2"	Create user with group
Multiple Options	./scale.sh 3 web	COUNT="\$1" SERVICE="\$2"	Scale service instances

⑤ BEST PRACTICES

- Always validate the number of arguments using ##.
- Provide helpful usage instructions.
- Use quotes around variables: "\$1"
- Never assume arguments exist.
- Use meaningful argument names in documentation.



⑥ ARGUMENT VALIDATION EXAMPLE

```
1 #!/bin/bash
2 if [ $# -lt 2 ]; then
3     echo "Usage: $0 <env> <region>"
4     echo "Example: $0 prod us-east-1"
5     exit 1
6 fi
7 ENV="$1"
8 REGION="$2"
9 echo "Deploying to $ENV in $REGION"
```



Explanation

- Ensures at least 2 arguments.
- Shows usage if missing.
- Prevents script from running with bad input.



REMEMBER:

Good scripts don't just work - they guide the user.
Validate input, handle errors, and make the experience smooth!



PRO TIP:

Combine arguments with flags for powerful CLI tools in production.



Instagram:
@veriqta



LinkedIn:
VERIQTA



X:
@veriqta



YouTube:
VERIQTA

VERIQTA



EXIT CODES TELL YOU WHETHER A COMMAND OR SCRIPT SUCCEEDED OR **FAILED**.
THEY ARE ESSENTIAL FOR **RELIABLE**, PRODUCTION-GRADE **AUTOMATION**.

① WHAT EXIT CODES MEAN

Every command or script returns a numeric status (exit code) to the shell after it finishes.



0

Success

The command or script completed successfully.



1-255

Failure

Something went wrong. The number gives a hint.



Did you know?

By convention, 0 means success.
Any non-zero value means failure.

② THE \$? VARIABLE

After any command, use `$?` to get its exit status.

```
$ ls /etc/passwd
$ echo $?
```

Command succeeded

```
$ ls /etc/passwd123
ls: cannot access '/etc/passwd123':
No such file or directory
$ echo $?
```

Command failed



Always check `$?` right after the command you care about.
Running another command will overwrite it.

③ SUCCESS VS FAILURE

EXIT CODE	MEANING	EXAMPLE
0	Success	<code>cp file1 file2</code> <code>echo \$?</code> 0
1	General error	Some command failed due to an unknown reason.
2	Misuse of shell builtins or command	<code>ls invalid_option</code> <code>echo \$?</code> 2
126	Command found but not executable	<code>./script.sh</code> (no execute perm) <code>echo \$?</code> 126
127	Command not found	<code>unknowncmd</code> <code>echo \$?</code> 127

★ Tip: Check `man bash` or specific command docs for more information on exit codes.

④ CUSTOM EXIT CODES

You can set custom exit codes in your scripts to represent specific outcomes.

```
# example: backup.sh
#!/bin/bash

if [ ! -d "$BACKUP_DIR" ]; then
  echo "Backup directory not found"
  exit 10      # Custom code: config error
fi

if tar -czf backup.tar.gz /data; then
  echo "Backup successful"
  exit 0      # Success
else
  echo "Backup failed"
  exit 20     # Custom code: backup failure
fi
```

EXAMPLE CODE MAP

0 = Success
10 = Config error
20 = Backup failed
30 = Network error
40 = Permission issue



Use meaningful exit codes so other tools (scripts, CI/CD, monitoring) can make smart decisions.

⑤ SCRIPT STATUS HANDLING

Handle exit codes to control script flow and error handling.

```
# Stop script if any command fails
set -e

# Check specific command
command_here
if [ $? -ne 0 ]; then
  echo "Command failed with exit code $?"
  exit 1
fi

echo "All good!"
```

Common Patterns

```
✓ if [ $? -eq 0 ]
  # success

✗ if [ $? -ne 0 ]
  # failure

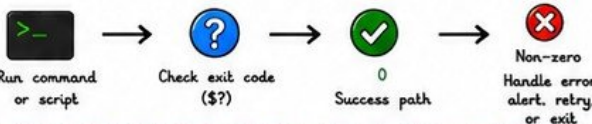
→ if command; then
  # run if success
else
  # run if failure
fi
```



BEST PRACTICES

- ✓ Always check `$?` for critical commands.
- ✓ Use `set -e` and `set -o pipefail`.
- ✓ Use meaningful custom exit codes.
- ✓ Log and act on failures quickly.
- ✓ Make your scripts fail fast and loudly.

REAL-WORLD EXAMPLE WORKFLOW



REMEMBER

A good script doesn't just run commands... it checks results!



Instagram:
@veriqta



LinkedIn:
VERIQTA



X:
@veriqta



YouTube:
VERIQTA

VERIQTA



7. CONDITIONAL STATEMENTS

CONDITIONAL STATEMENTS ALLOW SCRIPTS TO MAKE DECISIONS BASED ON CONDITIONS.
THE MAIN STRUCTURES ARE IF, ELIF (ELSE IF) AND ELSE.

① THE IF STATEMENT

Executes a block of code if a condition is true.

```
if [ condition ]; then
    # commands
fi
```

💡 In Bash, conditions inside [] are test expressions.

EXAMPLE

```
if [ -f /etc/passwd ]; then
    echo "File exists"
fi
```

✅ How it works:
Checks if /etc/passwd exists. If yes, it prints the message.

② IF...ELIF...ELSE STATEMENT

Use elif to check multiple conditions.

EXAMPLE

```
if [ condition1 ]; then
    # commands
elif [ condition2 ]; then
    # commands
else
    # commands
fi
```

```
read -p "Enter a number: " num
if [ $num -gt 10 ]; then
    echo "Greater than 10"
elif [ $num -eq 10 ]; then
    echo "Equal to 10"
else
    echo "Less than 10"
fi
```

⚙️ How it works:
The script checks the first condition. If false, it moves to elif. If all conditions fail, else runs.

③ ELSE STATEMENT

Runs when none of the conditions are true.

EXAMPLE

```
if [ condition ]; then
    # commands
else
    # commands
fi
```

```
if [ -d /backup ]; then
    echo "/backup exists"
else
    echo "/backup not found"
fi
```

📌 Use else to provide a default action.

④ NESTED CONDITIONS

Conditions inside another condition.

EXAMPLE

```
if [ condition1 ]; then
    if [ condition2 ]; then
        # commands
    else
        # commands
    fi
else
    # commands
fi
```

```
if [ -f "$FILE" ]; then
    if [ -s "$FILE" ]; then
        echo "File exists and not empty"
    else
        echo "File exists but is empty"
    fi
else
    echo "File does not exist"
fi
```

★ Use nesting for complex checks like multiple validations.

⑤ PRODUCTION DECISION LOGIC

Real-world examples of using conditionals in scripts.



1. CHECK DISK SPACE

Alert if disk usage > 80%

```
USAGE=$(df / | awk 'NR==2 {print $5}' | tr -d '%')
if [ $USAGE -gt 80 ]; then
    echo "Disk usage is ${USAGE}%"
    echo "ALERT: High disk usage!"
else
    echo "Disk usage is OK: ${USAGE}%"
fi
```

✅ Prevents outages due to full disks.



2. CHECK SERVICE STATUS

Restart service if not running

```
if systemctl is-active --quiet nginx; then
    echo "Nginx is running"
else
    echo "Nginx is NOT running."
    systemctl restart nginx
fi
```

✅ Auto-heal services in production.



3. CHECK USER

Create user if not present

```
USER="deploy"
if id "$USER" && /dev/null; then
    echo "User $USER exists"
else
    useradd -m -s /bin/bash $USER
    echo "User $USER created"
fi
```

✅ Idempotent user management.



4. VALIDATE FILE INPUT

Proceed only if file exists and is not empty

```
if [ ! -f "$FILE" ]; then
    echo "Error: File not found"
    exit 1
elif [ ! -s "$FILE" ]; then
    echo "Error: File is empty"
    exit 2
fi
```

✅ Fail fast with clear error messages.



5. ENVIRONMENT CHECK

Run different steps based on environment

```
ENV=$(ENV:-dev)
if [ "$ENV" = "prod" ]; then
    echo "Production mode"
elif [ "$ENV" = "staging" ]; then
    echo "Staging mode"
else
    echo "Development mode"
fi
```

✅ Safely handle dev, staging, and production.



BEST PRACTICES

- Always quote variables: "\$var"
- Use [] for portability, [[]] for advanced tests.
- Validate inputs before acting.
- Keep conditions simple and readable.
- Log decisions for easier troubleshooting.

COMMON TEST EXAMPLES

- | | | | |
|-----------|-------------------|-----------|--------------------------|
| -f FILE | File exists | -n STRING | String is not empty |
| -d DIR | Directory exists | -eq | Equal |
| -s FILE | File is not empty | -ne | Not equal |
| -z STRING | String is empty | -gt / -lt | Greater than / Less than |



QUICK TIP

Good condition logic is the foundation of reliable automation. Think before you act!



Instagram:
@veriqta



LinkedIn:
VERIQTA



X:
@veriqta



YouTube:
VERIQTA

VERIQTA



8. TEST OPERATORS

TEST OPERATORS ARE USED WITHIN [] OR [[]] TO CHECK CONDITIONS.

THEY HELP SCRIPTS MAKE DECISIONS BASED ON FILES, STRINGS, NUMBERS AND LOGIC.

① FILE TEST OPERATORS

Check properties of files and directories.

OPERATOR	MEANING	EXAMPLE	TRUE IF...
-e file	File exists	[-e /etc/passwd]	File exists
-f file	Regular file	[-f /etc/passwd]	Is a regular file
-d dir	Directory exists	[-d /var/log]	Is a directory
-s file	File is not empty	[-s file.txt]	Size > 0
-r file	Readable	[-r file.txt]	Readable by user
-w file	Writable	[-w file.txt]	Writable by user
-x file	Executable	[-x script.sh]	Executable by user
-u file	Setuid bit set	[-u file]	Setuid bit is set
-g file	Setgid bit set	[-g file]	Setgid bit is set
-k file	Sticky bit set	[-k dir]	Sticky bit is set
-nt file1 file2	file1 newer than file2	[file1 -nt file2]	file1 modified later
-ot file1 file2	file1 older than file2	[file1 -ot file2]	file1 modified earlier

★ TIP: Always quote paths with spaces: [-f "My File.txt"]

② STRING COMPARISONS

Compare text values.

OPERATOR	MEANING	EXAMPLE
= or ==	Equal	["\$a" = "\$b"]
!=	Not equal	["\$a" != "\$b"]
-z str	String is empty	[-z "\$str"]
-n str	String is not empty	[-n "\$str"]
str	String is not empty (informative)	["\$str"]

EXAMPLE

```
name="Ann"
if [ "$name" = "Ann" ]; then
    echo "Hello Ann"
elif [ -z "$name" ]; then
    echo "Name is empty"
else
    echo "Hello $name"
fi
```



Always use quotes around variables to avoid errors.

Use -z for empty, -n for not empty.

③ NUMERIC COMPARISONS

Compare numbers.

OPERATOR	MEANING	EXAMPLE
-eq	Equal	[\$a -eq \$b]
-ne	Not equal	[\$a -ne \$b]
-gt	Greater than	[\$a -gt \$b]
-ge	Greater or equal	[\$a -ge \$b]
-lt	Less than	[\$a -lt \$b]
-le	Less or equal	[\$a -le \$b]

EXAMPLE

```
cpu=85
if [ $cpu -gt 80 ]; then
    echo "High CPU!"
elif [ $cpu -ge 50 ]; then
    echo "CPU is moderate"
else
    echo "CPU is normal"
fi
```



Numbers must be integers.

For decimals, use bc or awk for comparison.

④ LOGICAL OPERATORS

Combine multiple conditions.

OPERATOR	MEANING	SYNTAX EXAMPLE
-a	AND (both true)	[cond1 -a cond2]
-o	OR (one true)	[cond1 -o cond2]
!	NOT (invert)	[! cond]

EXAMPLE

```
if [ -f "/etc/passwd" -a -r "/etc/passwd" ]; then
    echo "/etc/passwd exists and is readable"
fi
```

ALTERNATIVE (BETTER READABILITY)

```
# AND
if [ -f file ] && [ -r file ]; then
    echo "OK"
fi

# OR
if [ -d dir ] || [ -f dir ]; then
    echo "dir exists as dir or file"
fi

# NOT
if [ ! -f file ]; then
    echo "file not found"
fi
```

★ Use && and || for complex expressions (more reliable).

⑤ COMMON MISTAKES

① MISSING SPACES

✗ BAD
if [-f file.txt]; then
echo "Exists"
fi

✓ GOOD
if [-f file.txt]; then
echo "Exists"
fi

Spaces around [,], and operators are required. ✗

② MISSING QUOTES

✗ BAD
if [\$name = Ann]; then
echo "Hi"
fi

✓ GOOD
if ["\$name" = "Ann"]; then
echo "Hi"
fi

Always quote variables. 99

③ USING = INSTEAD OF -Eq

✗ BAD
if [\$a = \$b]; then
echo "Equal"
fi

✓ GOOD
if [\$a -eq \$b]; then
echo "Equal"
fi

= is for strings.
-eq is for numbers. =

④ FORGETTING \$?

✗ BAD
command
if [\$? = 0]; then
echo "Success"
fi

✓ GOOD
command
status=\$?
if [\$status -eq 0]; then
echo "Success"
fi

Capture exit status immediately after command. \$?

⑤ OVERLOOKING FILE PERMS

✗ BAD
if [-f file]; then
cat file
fi

✓ GOOD
if [-f file] && [-r file]; then
cat file
fi

Check readability before accessing a file. ✓



BEST PRACTICE

- Use tests to write safe and intelligent scripts.
- Validate everything before acting.
- Always handle edge cases.



QUICK REFERENCE

File?	[-f file]	Equal?	["\$a" = "\$b"]
Dir?	[-d dir]	Greater?	[\$a -gt \$b]
Empty?	[-z str]	Exists?	[-e path]



REMEMBER

A good engineer writes defensive scripts that expect things to go wrong.



Instagram:
@veriqta



LinkedIn:
VERIQTA



X:
@veriqta



YouTube:
VERIQTA

VERIQTA



CASE STATEMENTS ARE USED TO COMPARE A VALUE AGAINST MULTIPLE OPTIONS.
THEY ARE **CLEANER** AND EASIER TO READ THAN LONG IF...ELIF...ELSE CHAINS.

① CASE SYNTAX

The basic structure of a case statement.

```
case "$variable" in
  pattern1)
    commands ;;
  pattern2)
    commands ;;
  *)
    default commands ;;
esac
```

BREAKDOWN

- ✓ case: start
- ✓ variable: value to match
- ✓ pattern): match option
- ✓ ;;: end of block
- ✓ *): default case
- ✓ esac: end

② MULTIPLE OPTIONS

Group multiple options using | (pipe).

```
case "$cmd" in
  start|stop|restart)
    echo "Service command: $cmd" ;;
  status|stat)
    echo "Checking status..." ;;
  *)
    echo "Unknown command: $cmd" ;;
esac
```

EXAMPLE RUNS

- ./script.sh start
→ Service command: start
- ./script.sh status
→ Checking status...
- ./script.sh reload
→ Unknown command: reload

③ PATTERN MATCHING

Case patterns support wildcards.

PATTERN	MEANING	EXAMPLE MATCHES
*	Matches anything	any value
abc*	Starts with abc	abc, abcd, abc123
*xyz	Ends with xyz	xyz, testxyz, myxyz
a*b	Starts with a and ends with b	ab, acb, axxb
[abc]	Any one of a, b, or c	a, b, c
[!abc]	Anything except a, b, c	d, 1, z, etc.



Patterns make case very powerful for routing, validation, and automation logic.

④ MENU SCRIPT EXAMPLE

A simple menu-driven script using case.

```
1 echo "===== SYSTEM MENU ====="
2 echo "1) Start Service"
3 echo "2) Stop Service"
4 echo "3) Restart Service"
5 echo "4) Check Status"
6 echo "5) Exit"
7 read -p "Choose an option [1-5]: " opt
8
9 case "$opt" in
10 1) echo "Starting service..." ;;
11 2) echo "Stopping service..." ;;
12 3) echo "Restarting service..." ;;
13 4) echo "Service is running." ;;
14 5) echo "Goodbye!"; exit 0 ;;
15 *) echo "Invalid option!" ;;
16 esac
```

WHAT IT DOES

- ✓ Displays a menu
- ✓ Reads user choice
- ✓ Executes the selected action
- ✓ Handles invalid options safely

⑤ PRODUCTION EXAMPLES

Real-world use cases where case makes scripts cleaner and more reliable.



1. DEPLOYMENT ENVIRONMENT SELECTION

Choose environment for deployment.

```
case "$env" in
  dev) url="dev.api.com" ;;
  staging) url="staging.api.com" ;;
  prod) url="api.com" ;;
  *) echo "Invalid environment"; exit 1 ;;
esac
```



2. OS DETECTION

Run commands based on OS type.

```
case "$os" in
  ubuntu|debian) apt update ;;
  rhel|centos|amazon)
    yum install -y httpd ;;
  *) echo "Unsupported OS"; exit 1 ;;
esac
```



3. LOG LEVEL FILTERING

Handle different log levels.

```
case "$level" in
  INFO) echo "Show INFO logs" ;;
  WARN|WARNING) echo "Show WARN logs" ;;
  ERROR) echo "Show ERROR logs" ;;
  DEBUG) echo "Show DEBUG logs" ;;
  *) echo "Invalid log level" ;;
esac
```



4. USER ROLE HANDLING

Perform actions based on user role.

```
case "$role" in
  admin) echo "Full access granted" ;;
  devops) echo "Deployment access" ;;
  readonly) echo "Read-only access" ;;
  *) echo "Access denied"; exit 1 ;;
esac
```

⑥ BEST PRACTICES

- ✓ Use case when you have 3 or more discrete choices.
- ✓ Group similar options with | to reduce repetition.
- ✓ Always include a default case (*) for safety.
- ✓ Keep actions simple inside each block.
- ✓ Indent commands for readability.



COMMON MISTAKES



- ✗ Forgetting ;; at the end of a case block.
- ✗ Omitting the *) default case.
- ✗ Using case for simple true/false checks.
- ✗ Not quoting variables.
- ✗ Overusing patterns when exact match is better.



PRO TIP:

Case makes scripts easier to maintain, extend, and understand—especially in production.



LOOPS ALLOW SCRIPTS TO REPEAT TASKS AUTOMATICALLY.
THE MAIN LOOPS ARE **FOR**, **WHILE** AND **UNTIL**.

① FOR LOOPS

Used when you know how many times to repeat.

SYNTAX

```
for item in list; do
  commands
done
```

EXAMPLE

```
for i in 1 2 3 4 5; do
  echo "Number: $i"
done
```

★ Best for iterating over lists, files, users, services, etc.

③ UNTIL LOOPS

Runs until the condition becomes TRUE.

SYNTAX

```
until condition; do
  commands
done
```

EXAMPLE

```
count=1
until [ $count -gt 5 ]; do
  echo "Count: $count"
  count=$((count+1))
done
```

❗ Opposite of while loop. Condition is checked before each iteration.

② WHILE LOOPS

Used when you don't know how many times to repeat.

SYNTAX

```
while condition; do
  commands
done
```

EXAMPLE

```
count=1
while [ $count -le 5 ]; do
  echo "Count: $count"
  count=$((count+1))
done
```

✓ Repeats as long as the condition is TRUE.

④ LOOP CONTROL

Control how loops behave.

COMMAND	WORKS WITH	DESCRIPTION	EXAMPLE
break	for, while, until	Exit the loop immediately	break
continue	for, while, until	Skip to next iteration	continue
break N	Nested loops	Exit N levels of loops	break 2
continue N	Nested loops	Skip to next iteration N levels up	continue 2



Use **break** to stop early when condition is met.
Use **continue** to skip unwanted items.

⑤ PRODUCTION AUTOMATION EXAMPLES

1. RESTART ALL FAILED SERVICES

```
for svc in $(systemctl --failed --no-legend | awk '{print $1}'); do
  echo "Restarting $svc ..."
  sudo systemctl restart $svc
done
```



USE CASE

Automatically restart all failed services.

2. MONITOR DISK USAGE UNTIL SAFE

```
while true; do
  usage=$(df / | awk 'NR==2 {gsub("%", ""); print $5}');
  if [ "$usage" -lt 80 ]; then
    echo "Disk usage is safe: $usage%"
    break
  else
    echo "Disk usage high: $usage%"
    sleep 60
  fi
done
```



USE CASE

Keep checking disk usage until it drops below 80%.

3. WAIT UNTIL SERVICE BECOMES ACTIVE

```
until systemctl is-active --quiet myapp; do
  echo "Waiting for myapp service..."
  sleep 5
done
echo "myapp is now running!"
```



USE CASE

Wait for a service to be active before proceeding.

4. PROCESS LOG FILES

```
for file in /var/log/*.log; do
  echo "Processing $file"
  if [ -s "$file" ]; then
    gzip "$file"
    echo "Compressed $file"
  else
    echo "$file is empty, skipping"
  fi
done
```



USE CASE

Loop through log files, compress non-empty ones.

⑥ BEST PRACTICES

- ✓ Always define a clear exit condition for while and until loops.
- ✓ Avoid infinite loops.
- ✓ Use meaningful variable names.
- ✓ Test loops with small data first.
- ✓ Use `set -e` to fail fast on errors.
- ✓ Add logging inside long-running loops.



⑦ COMMON MISTAKES

- ✗ Forgetting **done**.
- ✗ Using `[ ]` instead of `[ [ ]` for complex conditions.
- ✗ Modifying the loop variable inside a for loop.
- ✗ Not updating the condition variable in while/until.
- ✗ Not quoting variables.
- ✗ Creating infinite loops without **break**.



⑧ QUICK REFERENCE

Loop Type	Use When...
for	You know the items or count
while	Repeat while condition is TRUE
until	Repeat until condition is TRUE
break	Exit the loop
continue	Skip to next iteration



REMEMBER:

Automation is about removing repetitive work.
Loops are the heart of every reliable automation script!



PRO TIP:

Combine loops with condition checks, functions and logging for powerful scripts.



Instagram:
@veriqta



LinkedIn:
VERIQTA



X:
@veriqta



YouTube:
VERIQTA

VERIQTA



FUNCTIONS HELP YOU ORGANIZE CODE, REDUCE REPETITION,
AND BUILD REUSABLE, MAINTAINABLE SCRIPTS.

① CREATING FUNCTIONS

Define a function with a name, parentheses, and a block.

```
function_name() {
    # commands
    return 0 # optional
}
```

EXAMPLE

```
greet() {
    echo "Hello, $1!"
}
greet "Ann"
# Output: Hello, Ann!
```

★ Call the function by its name like any other command.

③ RETURN VALUES

Functions return an exit status (0 = success by convention).

```
my_function() {
    # do something
    return 0 # success
    # return 1 # failure
}
```

EXAMPLE

```
check_user() {
    id "$1" &> /dev/null
    if [ $? -eq 0 ]; then
        echo "User exists"
        return 0
    else
        echo "User not found"
        return 1
    fi
}
```

Use the return value:

```
check_user "ann"
if [ $? -eq 0 ]; then
    echo "Proceed..."
else
    echo "Stop. User missing."
fi
```



IMPORTANT

A function can use **return** to send a status back to the caller using **\$?** .

② PARAMETERS

Functions can accept parameters, just like scripts.

SYMBOL	DESCRIPTION	EXAMPLE (INSIDE FUNCTION)
\$1, \$2, ... \$9	Positional parameters	echo "\$1, \$2"
##	Number of arguments	echo "Total: \$#"
\$@	All arguments (separate)	for i in "\$@"; do echo "\$i"; done
\$*	All arguments (single string)	echo "\$*"
\$0	Function name	echo "\$0"



Example with multiple parameters

```
add() {
    sum=$(( $1 + $2 ))
    echo "Sum: $sum"
}
add 5 7 # Output: Sum: 12
```

④ LOCAL VARIABLES

Variables inside functions are global by default.
Make them local to avoid conflicts.

EXAMPLE

```
my_func() {
    local var="value"
    local count=0
    # use $var, $count
}
```

```
count=100 # global
show_count() {
    local count=5
    echo "Inside: $count"
}
show_count
echo "Outside: $count"
# Inside: 5
# Outside: 100
```



Always use **local** for variables that should not affect the outside world.

⑤ REUSABLE SCRIPT DESIGN

Build modular functions to write cleaner, reusable and maintainable scripts.



1. SMALL & FOCUSED

Each function should do one thing and do it well.

- ✓ Validate input
- ✓ Check service
- ✓ Send alert



2. NAMING CONVENTION

Use clear, action-based function names.

- ✓ check_disk
- ✓ restart_service
- ✓ send_alert



3. DOCUMENT YOUR FUNCTIONS

Add comments to explain what the function does.

- ✓ Purpose
- ✓ Parameters
- ✓ Return codes



4. HANDLE ERRORS

Return meaningful codes and handle them.

- ✓ return 0 (success)
- ✓ return 1 (failure)
- ✓ Log the error



5. KEEP IT DRY

Don't repeat yourself. Reuse functions.

- ✓ Less code
- ✓ Easier to test
- ✓ Easier to update

⑥ FULL PRODUCTION EXAMPLE

```
#!/bin/bash
#----- Functions -----
log() {
    echo "[$(date +%F %T)] $1"
}
check_service() {
    local svc=$1
    if systemctl is-active --quiet "$svc"; then
        log "$svc is running"
        return 0
    else
        log "$svc is NOT running"
        return 1
    fi
}
#----- Main -----
svc=$1
if [ -z "$svc" ]; then
    echo "Usage: $0 <service>; exit 1"
fi
check_service "$svc"
if [ $? -ne 0 ]; then
    log "Attempting restart..."
    systemctl restart "$svc"
    check_service "$svc" || log "Restart failed!"
fi
```

RUN EXAMPLE

```
$ ./script.sh nginx
[2025-07-16 10:30:01]
nginx is running

$ ./script.sh mysql
[2025-07-16 10:30:10]
mysql is NOT running
[2025-07-16 10:30:10]
Attempting restart...
[2025-07-16 10:30:11]
mysql is running
```



TIP

Functions make long scripts easy to read, test, and maintain.

⑦ COMMON MISTAKES

- ✗ Forgetting parentheses
greet { ... } # Wrong
greet() { ... } # Correct
- ✗ Forgetting to call the function
my_func # Nothing happens
my_func() # Calls it
- ✗ Using global variables
Use **local** inside functions to avoid side effects.
- ✗ Ignoring return codes
Always check **\$?** after function calls.

⑧ QUICK REFERENCE

TOPIC	SYNTAX / USAGE
Define function	name() { commands }
Call function	name arg1 arg2
Parameters	\$1 \$2 ... \$9
All args	\$@ (separate)
Count args	##
Return value	return 0 / return 1
Check return	\$?
Local variable	local var=value



REMEMBER

Great scripts are not about length, they are about clarity, reuse, and reliability.



Instagram:
@veriqta



LinkedIn:
VERIQTA



X:
@veriqta



YouTube:
VERIQTA

VERIQTA 

12. FILE & DIRECTORY OPERATIONS

MASTERING FILE & DIRECTORY MANAGEMENT IS ESSENTIAL
FOR WRITING **SAFE**, **RELIABLE**, **PRODUCTION-READY** SCRIPTS.

1 CREATING FILES

- ★ Create empty file
- ★ Create file with content
- ★ Create multiple files

EXAMPLE

```
touch file.txt
echo "Hello" > file.txt
echo "Line" >> file.txt
touch {a,b,c}.log
```



TIP

Always quote variables when using paths.
`echo "$file" > "$dir/output.txt"`

2 COPYING FILES & DIRECTORIES

- ✓ Copy file
- ✓ Copy directory
- ✓ Preserve attributes
- ✓ Verbose copy

EXAMPLE

```
cp source.txt backup.txt
cp -r mydir/ mydir_backup/
cp -a /etc/nginx/ nginx_backup/
cp -av file.txt /backup/
```



NOTE

Use `-a` to preserve permissions, ownership, and timestamps.

3 MOVING & RENAMING

- 📁 Move file
- 📁 Move directory
- 📁 Rename file
- 📁 Rename directory

EXAMPLE

```
mv file.txt /tmp/
mv oldname.txt newname.txt
mv mydir/ /backup/
mv old_dir/ new_dir/
```



TIP

`mv` is atomic on the same filesystem.
Use it for safe deploys and rollbacks.

4 DELETING FILES SAFELY

- 🗑️ Delete file
- 🗑️ Delete directory
- 🗑️ Force delete
- 🗑️ Interactive delete

EXAMPLE

```
rm file.txt
rm -r old_dir/
rm -rf temp_dir/
rm -ri *.log
```



WARNING

Double-check paths before running `rm -rf`.
One wrong command can destroy data.

5 DIRECTORY VALIDATION

- ✓ Check if directory exists
- ✓ Create if not exists
- ✓ Check if writable

EXAMPLE

```
if [ -d "$dir" ]; then
    echo "Directory exists"
else
    mkdir -p "$dir"
fi

if [ -w "$dir" ]; then
    echo "Writable"
else
    echo "Not writable"
fi
```



TIP

Always validate paths before reading or writing files.

6 SAFE FILE HANDLING

- 🛡️ Check file exists
- 🛡️ Check file is not empty
- 🛡️ Check file is readable
- 🛡️ Check file size
- 🛡️ Avoid word splitting

EXAMPLE

```
if [ -f "$file" ]; then
    if [ -s "$file" ]; then
        cat "$file"
    else
        echo "File is empty"
    fi
else
    echo "File not found"
fi
```



TIP

Always use quotes: `"$var"`
Prevents bugs with spaces and special chars.

7 PRODUCTION EXAMPLES

📄 Rotate Logs

```
LOG_DIR="/var/log/myapp"
mkdir -p "$LOG_DIR/archive"
mv "$LOG_DIR/*.log" "$LOG_DIR/archive/"
"${date +%F}"
# keep last 7 days
find "$LOG_DIR/archive" -type f -mtime +7 -delete
```

📁 Backup Directory

```
SRC="/data/app"
DEST="/backup/app-${date +%F}"
cp -a "$SRC" "$DEST"
echo "Backup completed: $DEST"
```

8 COMMON MISTAKES

- ✗ Using `rm -rf` without checking path
- ✗ Not quoting variables
- ✗ Not validating directories
- ✗ Overwriting files accidentally
- ✗ Ignoring permissions
- ✗ Using relative paths in cron
- ✗ Not handling file errors
- ✗ Not checking available disk space



AVOID DISASTER

Think before you delete.
Validate before you write.
Automate only what is safe.

9 BEST PRACTICES

- ✓ Always validate inputs & paths
- ✓ Use absolute paths
- ✓ Quote all variables
- ✓ Check permissions
- ✓ Handle errors gracefully
- ✓ Keep backups
- ✓ Log important actions
- ✓ Test in a safe environment first
- ✓ Document your scripts



REMEMBER

Good file operations prevent data loss and keep your scripts reliable in production.



Instagram:
@veriqta



LinkedIn:
VERIQTA



X:
@veriqta



YouTube:
VERIQTA

VERIQTA ❤️

TEXT PROCESSING COMMANDS ALLOW YOU TO **FILTER**, **TRANSFORM**,
AND **ANALYZE** TEXT EFFICIENTLY IN PRODUCTION.

① ESSENTIAL TEXT PROCESSING COMMANDS

COMMAND	PURPOSE	BASIC SYNTAX	EXAMPLE
grep	Search for patterns	grep [options] pattern file	grep "ERROR" app.log
sed	Stream editor (find & replace)	sed 's/old/new/g' file	sed 's/ERROR/WARN/g' app.log
awk	Pattern scanning & processing	awk '{print \$1}' file	awk -F, '{print \$1,\$3}' data.csv
cut	Extract columns from text	cut -d',' -f1 file	cut -d',' -f1 data.csv
sort	Sort lines	sort [options] file	sort -u users.txt
uniq	Remove or report duplicates	uniq [options] file	sort file uniq -c
tr	Translate or delete characters	tr 'a-z' 'A-Z' < file	tr -d '\r' < file.txt
wc	Count lines, words, characters	wc [options] file	wc -l app.log

② PRODUCTION LOG FILTERING EXAMPLES

- Find ERROR lines
`grep "ERROR" app.log` # show all ERROR lines
- Find ERROR with line numbers
`grep -n "ERROR" app.log` # with line numbers
- Find ERROR and WARNING
`grep -E "ERROR|WARN" app.log` # multiple patterns
- Show last 100 ERROR lines
`grep "ERROR" app.log | tail -100` # recent errors
- Count ERROR occurrences
`grep -c "ERROR" app.log` # count matches
- Exclude noisy patterns
`grep -v "DEBUG" app.log` # exclude DEBUG
- Find IP addresses in logs
`grep -Eo '([0-9]{1,3}\.){3}([0-9]{1,3})' app.log` # extract IPs

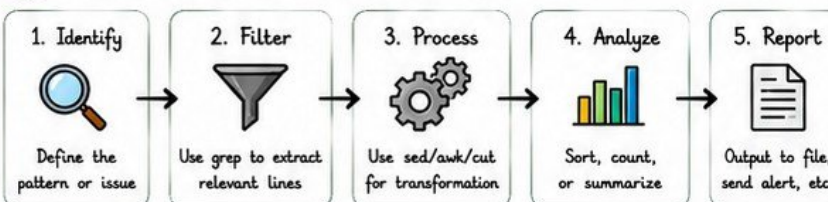
③ COMMAND EXAMPLES

grep 	<code>grep "timeout" app.log</code> <code>grep -i "error" app.log</code> <code>grep -A 3 "failed" app.log</code> <code>grep -nE "ERROR WARN" app.log</code>	sort 	<code>sort file.txt</code> <code>sort -r file.txt</code> <code>sort -u file.txt</code> <code>sort -nk2 data.txt</code>
sed 	<code>sed 's/old/new/g' file.txt</code> <code>sed -i 's/ERROR/WARN/g' app.log</code> <code>sed -n '10,20p' file.txt</code>	uniq 	<code>sort file.txt uniq</code> <code>sort file.txt uniq -d</code> <code>sort file.txt uniq -c</code>
awk 	<code>awk '{print \$1}' file.txt</code> <code>awk -F, '{print \$2}' data.csv</code> <code>awk '\$3 > 100 {print}' data.csv</code>	tr 	<code>tr 'a-z' 'A-Z' < file.txt</code> <code>tr -d '\r\n' < file.txt</code> <code>tr ' ' '\t' < file.txt</code>
cut 	<code>cut -d',' -f1,3 data.csv</code> <code>cut -d':' -f1 /etc/passwd</code>	wc 	<code>wc -l file.txt</code> <code>wc -w file.txt</code> <code>wc -c file.txt</code>

④ BEST PRACTICES

-  **Filter early, process less**
Use grep to reduce data before heavy processing.
-  **Use exact patterns**
Be specific to avoid unnecessary matches.
-  **Quote variables**
Always quote variables to prevent word splitting.
-  **Handle large files carefully**
Use tools that stream line by line (grep, awk, sed).
-  **Use -i for case-insensitive**
Useful for unknown casing in logs.
-  **Combine commands**
Pipes (|) are powerful, but keep it readable.

⑤ PRODUCTION LOG PROCESSING WORKFLOW



TIP Test commands on **small** samples first, then apply to large logs.

⑥ COMMON MISTAKES

- ✗ Forgetting quotes → pattern expands or breaks
- ✗ Using grep without -i for mixed case
- ✗ Not handling spaces in file paths
- ✗ Overusing pipes → hard to debug
- ✗ Not testing commands before automating



REMEMBER

Small, simple commands done well beat complex commands done poorly.

14. ERROR HANDLING

GOOD ERROR HANDLING TURNS UNKNOWN FAILURES INTO CLEAR, ACTIONABLE SIGNALS.

1 UNDERSTANDING FAILURES

- ★ Commands can fail for many reasons.
- ★ Always check exit status (\$?).
- ★ Non-zero exit codes indicate failure.
- ★ Never ignore failures in production.



TIP

Always assume a command can fail. Handle it like a professional.

EXIT STATUS EXAMPLES

EXIT CODE	MEANING
0	Success
1	General Error
2	Misuse of shell builtin
126	Permission denied
127	Command not found
128 + N	Terminated by signal N

2 EXIT ON ERROR (set -e)

Stop script immediately if any command fails.

EXAMPLE

```
#!/bin/bash
set -e # exit immediately on error

echo "Step 1"
rm /important/file
echo "Step 2" # will not run if rm fails
```



NOTE

Use with care. Some commands may return non-zero even when acceptable.

3 UNDEFINED VARIABLES (set -u)

Exit when using an undefined variable.

EXAMPLE

```
#!/bin/bash
set -u # treat unset variables as error

echo "User: $USER"
echo "Home: $NOT_DEFINED" # error if not set
```



TIP

Prevents bugs caused by typos or missing values.

4 PIPE FAILURES (set -o pipefail)

Catch failures in any part of a pipeline.

EXAMPLE

```
#!/bin/bash
set -o pipefail

cat file.txt | grep "ERROR" | sort
echo "Pipeline exit code: $?"
```



NOTE

Without pipefail, the pipeline returns the last command's exit code.

5 TRAP COMMAND

Run custom actions when script exits or fails.

EXAMPLE

```
#!/bin/bash
trap 'echo "Error on line $LINENO. Exit code: $?"' ERR
trap 'echo "Script interrupted"; cleanup' INT TERM

echo "Running..."
false # triggers ERR trap
```

COMMON TRAPS

EXIT	Script exits
ERR	Command fails
INT	Ctrl+C
TERM	Termination
HUP	Terminal closed

6 CLEANUP FUNCTIONS

Always clean up temporary files and resources.

EXAMPLE

```
#!/bin/bash
set -euo pipefail

TMP_FILE=$(mktemp)

cleanup() {
  rm -f "$TMP_FILE"
  echo "Cleaned up."
}

trap cleanup EXIT

# main script work here
echo "Using $TMP_FILE"
```



TIP

Use trap with cleanup to avoid leaving junk behind.



CLEAN ENV.
Happy Ops!

7 RETRY LOGIC

Retry commands that may fail due to transient issues.

EXAMPLE: RETRY FUNCTION

```
retry() {
  local n=0 max=3 delay=5
  until "$@"; do
    n=$((n+1))
    if [ $n -ge $max ]; then
      echo "Failed after $max attempts."
      return 1
    fi
    echo "Attempt $n failed. Retrying in $delay sec..."
    sleep $delay
  done
}
```

USAGE

```
retry curl -fsS
https://example.com/health
```



Great for network calls and API checks.

8 PRODUCTION DEBUGGING

Tools & techniques for faster debugging.

- ★ Use set -x for command tracing.
- ★ Print variables with echo.
- ★ Use PS4 for better trace logs.
- ★ Redirect output for analysis.
- ★ Reproduce with small test cases.

EXAMPLE

```
#!/bin/bash
set -x
PS4='+ ${BASH_SOURCE}:${LINENO}: '
echo "Debug mode ON"
./script.sh 2>debug.log
```

9 ERROR HANDLING BEST PRACTICES

- ✓ Always use: set -euo pipefail
- ✓ Validate inputs and dependencies.
- ✓ Check critical command exit codes.
- ✓ Use traps for cleanup and safety.
- ✓ Log meaningful error messages.
- ✓ Fail fast, fail clean.
- ✓ Handle expected errors gracefully.
- ✓ Test error paths, not just happy path.
- ✓ Document error flows in your scripts.



REMEMBER

Good error handling is not about avoiding errors. It's about controlling them. Make failures predictable, visible, and recoverable.



Instagram:
@veriqta



LinkedIn:
VERIQTA



X:
@veriqta



YouTube:
VERIQTA

VERIQTA 

15. LOGGING & DEBUGGING

GOOD LOGGING HELPS YOU SEE WHAT HAPPENS.
GOOD DEBUGGING HELPS YOU FIND WHY IT HAPPENED.

① LOGGING FUNDAMENTALS

- ★ Log important events.
- ★ Log context and results.
- ★ Make logs easy to read.
- ★ Store logs in the right place.
- ★ Never log sensitive data.

EXAMPLE

```
#!/bin/bash
LOG_FILE="/var/log/my_script.log"
echo "Script started" >> "$LOG_FILE"
echo "User: $USER" >> "$LOG_FILE"
echo "Task completed" >> "$LOG_FILE"
```

② TIMESTAMPS

Always add timestamps to your logs.

EXAMPLE

```
#!/bin/bash
log() {
  echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" >> "$LOG_FILE"
}
log "Backup started"
```

TIP
Consistent timestamps make logs searchable and useful.

③ LOG LEVELS

Use log levels to show importance.

LEVEL	USE WHEN	EXAMPLE
DEBUG	Detailed info for troubleshooting	log "DEBUG: value is \$val"
INFO	General operational messages	log "INFO: process started"
WARN	Something unexpected happened	log "WARN: disk space low"
ERROR	Something failed	log "ERROR: backup failed"
FATAL	Critical failure, script cannot continue	log "FATAL: exiting now"

④ DEBUG MODE (set -x)

Print each command before it executes.

EXAMPLE

```
#!/bin/bash
set -x # turn on debug mode
echo "Running..."
ls -l /tmp
set +x # turn off debug mode
echo "Done"
```

NOTE
Use only when debugging. Remove or disable in production.

⑤ REDIRECTING OUTPUT

Control where output goes.

EXAMPLE

```
./script.sh > output.log # stdout
./script.sh 2> error.log # stderr
./script.sh >> output.log 2>&1 # both to same file
./script.sh | tee output.log # show + save
```

TIP
Use 2>&1 to capture both output and errors.

⑥ ERROR LOGS

Keep errors separate and easy to find.

EXAMPLE

```
#!/bin/bash
LOG_FILE="/var/log/my_script.log"
ERR_FILE="/var/log/my_script_error.log"
some_command >> "$LOG_FILE" 2>> "$ERR_FILE"
```

TIP
Monitor error logs regularly. They tell you what broke.

⑦ STRUCTURED LOGGING

Log in a consistent, parseable format.

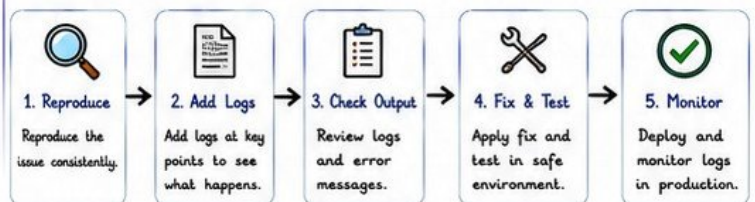
EXAMPLE (KEY=VALUE)

```
log() {
  local level="$1"; shift
  echo "time=$(date -Iseconds) level=$level user=$USER msg=\"${*}\" >> \"$LOG_FILE"
}
log INFO "User login successful"
```

WHY?
Structured logs are easier to search, filter, and analyze automatically.

⑧ DEBUGGING WORKFLOW

A simple process to find issues.



⑨ PRODUCTION LOGGING TIPS

- ✓ Log what matters. Avoid noise.
- ✓ Use log levels consistently.
- ✓ Include timestamps.
- ✓ Include context (user, IDs, paths).
- ✓ Never log passwords or secrets.
- ✓ Rotate or clean up old logs.
- ✓ Keep logs on reliable storage.
- ✓ Monitor logs and set alerts.
- ✓ Use structured logs when possible.
- ✓ Document your logging approach.



REMEMBER

Good logging helps you answer:

- What happened?
- When did it happen?
- Where did it happen?
- Why did it happen?

FILES ARE AT THE HEART OF AUTOMATION.
HANDLE THEM SAFELY, EFFICIENTLY AND RELIABLY.

1 READING FILES

- ★ Read entire file
- ★ Read line by line
- ★ Read specific fields
- ★ Skip empty lines



TIP

Use `IFS= read -r` to prevent trimming and backslash issues.

EXAMPLE

```
# Entire file
cat file.txt

# Line by line
while IFS= read -r line; do
    echo "$line"
done < file.txt

# Read first field
awk '{print $1}' file.txt

# Skip empty lines
grep -v '^$' file.txt
```

2 WRITING FILES

- ★ Overwrite file
- ★ Write multiple lines
- ★ Write formatted data



NOTE

Use `>` to overwrite and `>>` to append.

EXAMPLE

```
# Overwrite
echo "Hello" > file.txt

# Multiple lines
cat > file.txt <<'EOF'
Line 1
Line 2
EOF

# Formatted output
printf "%-10s %s\n" "Name"
"Status" > report.txt
```

3 APPENDING DATA

- ★ Append single line
- ★ Append multiple lines
- ★ Append with timestamp

EXAMPLE

```
echo "New line" >> file.txt

cat <<'EOF' >> file.txt
Appended line 1
Appended line 2
EOF

echo "[$(date +%F %T)] Backup done"
>> backup.log
```

4 PROCESSING CSV FILES

- ★ Read CSV line by line
- ★ Access columns
- ★ Skip header
- ★ Use `IFS=','`



TIP

Always quote variables when using `IFS`.

EXAMPLE

```
# Skip header and read CSV
tail -n +2 data.csv |
while IFS=',' read -r col1 col2 col3; do
    echo "Col1=$col1, Col2=$col2, Col3=$col3"
done

# Process and filter
awk -F',' ' $3 > 100 {print $0}' data.csv
```

5 CONFIGURATION FILES

- ✓ Store key=value pairs
- ✓ Source configuration
- ✓ Validate required keys



NOTE

Use `set -a` to automatically export variables from config.

EXAMPLE

```
# config.env
DB_HOST=localhost
DB_USER=admin
DB_PASS=secret

# Load config
set -a
source config.env
set +a
echo "Database: $DB_HOST"
```

6 TEMPORARY FILES

- ★ Create temp file
- ★ Create temp directory
- ★ Auto cleanup



TIP

Always clean up temporary files to avoid disk usage.

EXAMPLE

```
TMP_FILE=$(mktemp)
echo "data" > "$TMP_FILE"

TMP_DIR=$(mktemp -d)
cd "$TMP_DIR"

# Cleanup
rm -f "$TMP_FILE"
rm -rf "$TMP_DIR"
```

7 FILE PERMISSIONS

- ★ Check permissions
- ★ Change permissions
- ★ Set ownership

EXAMPLE

```
ls -l file.txt
chmod 640 file.txt
chown user:group file.txt
```

COMMON PERMISSION VALUES

PERMISSION	MEANING
644	Read/Write for owner, Read for group & others
755	Read/Write/Execute for owner, Read/Execute for others
600	Read/Write for owner only
700	Read/Write/Execute for owner only



8 FILE LOCKING CONCEPTS

- ★ Prevent multiple processes from writing at the same time
- ★ Use `flock` for simple locking
- ★ Avoid data corruption



NOTE

`flock` is simple and reliable for most shell scripts.

EXAMPLE

```
exec 200> /var/lock/my_script.lock
flock -n 200 || { echo "Another instance is running"; exit 1; }

# Critical section
echo "Running task..."
sleep 30

# Lock released automatically
exec 200>&-
```



REMEMBER

- ✓ Always validate file existence before using.
- ✓ Handle files safely.
- ✓ Clean up temporary data.
- ✓ Log meaningful information.
- ✓ Permissions matter for security.

9 PRODUCTION EXAMPLES



1. LOG ROTATION

```
mv app.log app.log.$(date +%F)
touch app.log
```



2. SAFE CONFIG UPDATE

```
cp config.env config.env.bak
echo "NEW_KEY=value" >> config.env
```



3. CSV REPORT GENERATION

```
echo "Name,Status" > report.csv
echo "App1,OK" >> report.csv
echo "App2,FAIL" >> report.csv
```



4. TEMP WORKFLOW

```
TMP_DIR=$(mktemp -d)
# do work
rm -rf "$TMP_DIR"
```



5. APPEND WITH TIMESTAMP

```
echo "[$(date +%F %T)] Job completed" >> job.log
```



Instagram:
@veriqta



LinkedIn:
VERIQTA



X:
@veriqta



YouTube:
VERIQTA



AUTOMATION TURNS REPEATED TASKS INTO RELIABLE, CONSISTENT AND REUSABLE SOLUTIONS.

1 BACKUP AUTOMATION

- ★ Backup files & directories
- ★ Compress backups
- ★ Rotate old backups
- ★ Verify backup success



TIP

Always verify backups and test restores regularly.

EXAMPLE

```
#!/bin/bash
SRC="/data/app"
DEST="/backup/app_$(date +%F)"
mkdir -p "$DEST"
tar -czf "$DEST.tar.gz" "$SRC"
echo "Backup completed: $DEST.tar.gz"

# Remove backups older than 7 days
find /backup -type f -mtime +7 -name "*.gz" -exec rm -f {} \;
```

2 CLEANUP AUTOMATION

- ✓ Remove temp files
- ✓ Clear old logs
- ✓ Clean package cache
- ✓ Free disk space



NOTE

Automate cleanup to prevent disk full and performance issues.

EXAMPLE

```
#!/bin/bash
# Remove files older than 3 days
find /tmp -type f -mtime +3 -delete
# Clear old logs
find /var/log -type f -name "*.log" -mtime +7 -delete
# Clean package cache
apt-get clean
echo "Cleanup completed"
```

3 HEALTH CHECK SCRIPTS

- ♥ Check services
- ♥ Verify ports
- ♥ Validate endpoints
- ♥ Return clear status

EXAMPLE

```
#!/bin/bash
SERVICE=nginx
if systemctl is-active --quiet $SERVICE; then
    echo "OK: $SERVICE is running"
    exit 0
else
    echo "CRITICAL: $SERVICE is down"
    exit 1
fi
```



TIP

Exit codes: 0 = OK, 1 = WARNING, 2 = CRITICAL
Perfect for monitoring systems.

4 MONITORING SCRIPTS

- 📊 Check disk usage
- 📊 Check memory usage
- 📊 Check CPU load
- 📊 Collect metrics

EXAMPLE

```
#!/bin/bash
df -h | awk 'NR==2 {print "Disk: " $5}'
free -m | awk 'NR==2 {printf "Mem: %dX\n", $3/$2*100}'
uptime | awk -Fload average: '{print "CPU Load: " $2}'
echo "---- $(date) ----" >> /var/log/monitor.log
```



TIP

Send output to logs or monitoring systems (Prometheus, Datadog, etc).

5 DEPLOYMENT HELPERS

- 🚀 Pull latest code
- 🚀 Install dependencies
- 🚀 Run migrations
- 🚀 Start/Reload services

EXAMPLE

```
#!/bin/bash
git pull origin main
npm install --production
./migrate.sh
systemctl reload myapp
echo "Deployment completed"
```



NOTE

Idempotent scripts make deployments safe to run multiple times.

6 SERVICE RESTART AUTOMATION

- 🔄 Restart services
- 🔄 Handle failures
- 🔄 Retry if needed
- 🔄 Notify on failure

EXAMPLE

```
#!/bin/bash
SERVICE=nginx
systemctl restart $SERVICE
if [ $? -ne 0 ]; then
    echo "Failed to restart $SERVICE" |
    mail -s "ALERT: $SERVICE Restart Failed" admin@example.com
    exit 1
fi
echo "$SERVICE restarted successfully"
```

7 ALERT SCRIPTS

- 🔔 Send email alerts
- 🔔 Send Slack alerts
- 🔔 Send webhook alerts
- 🔔 Custom notifications

EXAMPLE

```
#!/bin/bash
SUBJECT="Disk Space Alert"
MESSAGE="Disk usage is above 90% on $(hostname)"
echo "$MESSAGE" | mail -s "$SUBJECT" admin@example.com
curl -X POST -H 'Content-type: application/json' \
--data '{"text": "$SUBJECT - $MESSAGE"}' \
https://hooks.slack.com/services/XXXX/XXXX/XXXX
```



TIP

Keep alerts actionable and avoid alert fatigue.

8 PRODUCTION WORKFLOW

A typical automation script workflow.



PRODUCTION SCRIPTING PRINCIPLES

- ✓ Automate repeatable tasks.
- ✓ Fail fast and exit clearly.
- ✓ Log everything important.
- ✓ Validate inputs and outputs.
- ✓ Keep scripts idempotent.
- ✓ Handle errors gracefully.
- ✓ Test before automation.
- ✓ Monitor and alert issues.
- ✓ Document your scripts.



REMEMBER

Great automation saves time, prevents human error, and keeps systems reliable.



Instagram:
@veriqta



LinkedIn:
VERIQTA



X:
@veriqta



YouTube:
VERIQTA

VERIQTA 

18. CRON JOBS & SCHEDULING

CRON RUNS TASKS AUTOMATICALLY AT SCHEDULED TIMES.
RELIABLE SCHEDULING KEEPS SYSTEMS HEALTHY AND AUTOMATED.

① CRON BASICS

- ★ Cron is a time-based job scheduler.
- ★ Runs tasks in the background.
- ★ Uses crontab files to store schedules.
- ★ Runs as the user who owns the crontab.



TIP

Cron is perfect for automation, backups, cleanup, monitoring, and reporting.

CRON DAEMON



crond checks schedules every minute and runs due jobs.

③ SCHEDULING JOBS

- 📁 List current crontab
- 📁 Edit crontab
- 📁 Install new crontab
- 📁 Remove all jobs
- 📁 View system-wide crontab

EXAMPLE COMMANDS

```
crontab -l # List jobs
crontab -e # Edit jobs
crontab myjobs # Install from file
crontab -r # Remove all jobs
sudo crontab -l # System-wide jobs
```



Each user has their own crontab.
System jobs are in /etc/crontab or /etc/cron.d/

⑤ LOGGING CRON OUTPUT

- ✓ Cron emails output to the user by default.
- ✓ Redirect output to log files for better tracking.
- ✓ Always log both STDOUT and STDERR.

EXAMPLE

```
0 2 * * * /script.sh >> /var/log/script.log 2>&1
*/15 * * * * /health.sh >> /var/log/health.log 2>&1
0 3 * * * /backup.sh > /dev/null 2>&1
```



TIP

Use timestamps inside logs to understand when the job actually ran.
echo "[\$(date +%F %T)] Job started" >> /var/log/job.log

⑦ DEBUGGING SCHEDULED JOBS

- 🔍 Run script manually
- 🔍 Check logs
- 🔍 Add debug output
- 🔍 Check permissions
- 🔍 Check cron logs

EXAMPLE WORKFLOW

```
# 1. Run manually
bash /path/to/script.sh
# 2. Check logs
tail -f /var/log/script.log
# 3. Add debug
set -x
exec >> /var/log/script_debug.log 2>&1
# 4. Check cron logs
grep CRON /var/log/syslog # or /var/log/cron
```



TIP

Use scripts with the same user and environment as cron.

② CRONTAB SYNTAX

Cron uses 5 time fields followed by the command.

*	*	*	*		command_to_run
Min	Hour	Day	Month	Day of Week	Command

FIELD VALUES

Minute	0 - 59
Hour	0 - 23
Day of Month	1 - 31
Month	1 - 12 (or names: Jan, Feb, ...)
Day of Week	0 - 7 (0 and 7 are Sunday)

EXAMPLES

```
0 2 * * * /script.sh
# Every day at 2 AM

*/15 * * * * /script.sh
# Every 15 minutes

0 1 * * 0 /backup.sh
# Every Sunday at 1 AM
```

④ ENVIRONMENT VARIABLES

- ★ Cron runs with a minimal environment.
- ★ Always set necessary variables in your script or crontab.
- ★ Define PATH, HOME, SHELL, and others.

EXAMPLE

```
# In crontab:
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:\
/usr/sbin:/usr/bin:/sbin:/bin
HOME=/home/user
0 2 * * * /home/user/backup.sh
```



NOTE

Always use full paths in scripts and commands.
Do not rely on relative paths.

⑥ COMMON CRON ISSUES

- ✗ Script not running
- ✗ Wrong paths
- ✗ Missing permissions
- ✗ Environment issues
- ✗ Silent failures
- ✗ Output not logged

CAUSE

Wrong syntax or file not saved
Use full paths
Script/file not executable or readable
Missing PATH, variables
Errors not logged
No redirection or mail disabled

⑧ PRODUCTION SCHEDULING PRACTICES

- ✓ Keep scripts idempotent.
- ✓ Use full paths always.
- ✓ Log everything.
- ✓ Set environment explicitly.
- ✓ Alert on failures.
- ✓ Review schedules regularly.

BEST PRACTICES EXAMPLE

```
# Clean and safe job
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOME=/home/user
MAILTO=admin@example.com

0 2 * * * /usr/local/bin/backup.sh >> /var/log/backup.log 2>&1
15 * * * * /usr/local/bin/health.sh >> /var/log/health.log 2>&1
0 3 * * 0 /usr/local/bin/report.sh >> /var/log/report.log 2>&1
```



REMEMBER

Good scheduling is not just about running on time, it's about reliability, visibility and recoverability.

CRON CHEAT SHEET

```
Every minute: * * * * * command
Every 5 min: */5 * * * * command
Every hour: 0 * * * * command
Every day at 2 AM: 0 2 * * * command
Every Sunday at 1 AM: 0 1 * * 0 command
```



KEY TAKEAWAY

A well-written cron job runs quietly, predictably and recovers gracefully even when things go wrong.



Instagram:
@veriqta



LinkedIn:
VERIQTA



X:
@veriqta



YouTube:
VERIQTA

VERIQTA



GOOD HABITS TODAY = RELIABLE, SECURE AND MAINTAINABLE
SCRIPTS TOMORROW.

① DEFENSIVE SCRIPTING

- ★ Fail fast and loudly.
- ★ Handle errors explicitly.
- ★ Use strict mode.
- ★ Quote variables.
- ★ Check assumptions.

EXAMPLE

```
#!/bin/bash
set -euo pipefail
IFS=$'\n\t'

trap 'echo "ERROR on line $LINENO" >&2' ERR
echo "Script started"
```



TIP

Defensive scripts fail early, provide clear messages, and prevent damage.

③ SECURITY CONSIDERATIONS

- 💜 Use least privilege.
- 💜 Avoid hardcoding secrets.
- 💜 Protect sensitive data.
- 💜 Check file permissions.
- 💜 Beware of command injection.

EXAMPLE

```
# Bad (hardcoded secret)
PASS="MySecret123"

# Good (use environment variable)
PASS="${DB_PASS:-?Missing DB_PASS}"

# Quote variables
rm -f "$file"
```



NOTE

Security is not optional. Small mistakes can cause big incidents.

⑤ IDEMPOTENT SCRIPTS

- ✓ Running multiple times should be safe.
- ✓ Check before creating, deleting or updating.
- ✓ Avoid duplicate changes.

EXAMPLE

```
if [ ! -d "/opt/myapp" ]; then
  mkdir -p "/opt/myapp"
  echo "Directory created"
else
  echo "Directory already exists"
fi
```



NOTE

Idempotent scripts are safe, repeatable and automation-friendly.

⑦ PERFORMANCE OPTIMIZATION

- ⚡ Avoid useless subshells.
- ⚡ Use built-ins.
- ⚡ Minimize external commands.
- ⚡ Use efficient loops.
- ⚡ Cache expensive operations.

EXAMPLE

```
# Bad (useless cat)
cat file.txt | while read line; do
  echo "$line"
done

# Good (redirect instead)
while read line; do
  echo "$line"
done < file.txt
```



TIP

Small optimization improves script speed, especially on large data and frequent executions.

⑨ VERSION CONTROL

- 🔗 Store scripts in Git.
- 🔗 Commit early and often.
- 🔗 Use meaningful messages.
- 🔗 Tag releases.
- 🔗 Review changes.

EXAMPLE COMMANDS

```
git init
git add script.sh
git commit -m "Add backup script"
git tag -a v1.0 -m "Initial release"
git push origin main --tags
```



TIP

Version control provides history, collaboration and rollback capability.

② INPUT VALIDATION

- ✓ Validate all user input.
- ✓ Check required arguments.
- ✓ Validate file paths.
- ✓ Use patterns and ranges.

EXAMPLE

```
if [[ $# -lt 1 ]]; then
  echo "Usage: $0 <file>" >&2
  exit 1
fi

file="$1"
if [[ ! -f "$file" ]]; then
  echo "File not found: $file" >&2
  exit 1
fi
```



NOTE

Never trust user input. Validate, sanitize and check before using it.

④ PORTABILITY

- ✓ Write POSIX-compliant code.
- ✓ Avoid bash-only features when not needed.
- ✓ Use /usr/bin/env bash for portability.
- ✓ Test on different systems.

EXAMPLE

```
#!/usr/bin/env bash

# Use POSIX syntax
if [ -f "$file" ]; then
  echo "Exists"
fi

# Avoid [[ ]] if not needed
[ "$count" -gt 10 ]
```



TIP

Portable scripts run everywhere with fewer unexpected failures.

⑥ MODULAR DESIGN

- 🔗 Break code into functions.
- 🔗 Keep functions small and focused.
- 🔗 Reuse code.
- 🔗 Avoid repetition.

EXAMPLE

```
function log() {
  echo "[$(date +%F %T)] $*"
}

function check_service() {
  systemctl is-active --quiet "$1"
}
```



TIP

Modular code is easier to read, test and maintain.

⑧ DOCUMENTATION

- 📖 Use meaningful comments.
- 📖 Explain what and why, not how.
- 📖 Document usage.
- 📖 Keep docs updated.

EXAMPLE

```
#!/bin/bash
# Script: backup.sh
# Purpose: Backup application data
# Usage: backup.sh <source_dir>
# Author: VERIQTA
# Date: 2025-07-16

# Backup source directory
backup_dir="$1"
```



NOTE

Good docs save time, prevent errors and help your future self.

⑩ PRODUCTION CHECKLIST

- ✓ Script has input validation
- ✓ Script uses strict mode (set -euo pipefail)
- ✓ All errors are handled
- ✓ Logs are meaningful
- ✓ Sensitive data is not hardcoded
- ✓ Script is idempotent
- ✓ Script is portable
- ✓ Permissions are set correctly
- ✓ Tested in staging environment
- ✓ Documentation is complete
- ✓ Stored in version control
- ✓ Ready for production



REMEMBER

Production-grade scripts are built with care, tested with discipline, and maintained with pride. Quality scripting is a force multiplier.



KEY TAKEAWAY

The best scripts are easy to read, safe to run, easy to maintain and built to scale. Follow these practices and your scripts will never betray you.



Instagram:
@veriqta



LinkedIn:
VERIQTA



X:
@veriqta



YouTube:
VERIQTA

VERIQTA





VERIQTA

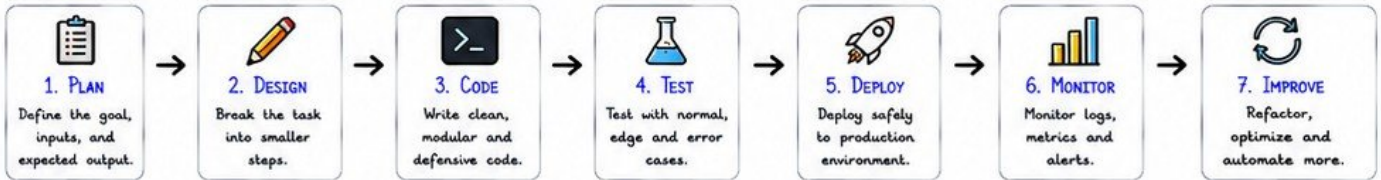
20. VOLUME 1 PRODUCTION REVIEW

VOLUME 1

PAGE 20 OF 20

YOU NOW HAVE THE FOUNDATIONS TO BUILD RELIABLE,
SAFE AND PRODUCTION-READY SHELL SCRIPTS.

① END-TO-END SHELL SCRIPTING WORKFLOW



② ESSENTIAL COMMAND CHEAT SHEET

COMMAND	PURPOSE	COMMAND	PURPOSE
ls -lah	List files (detailed)	date	Show date and time
cd /path	Change directory	cal	Show calendar
pwd	Print working directory	echo \$VAR	Print variable
cp -r src/ dest/	Copy directory	cat file	View file content
mv old new	Move or rename	less file	View large file
rm -rf path	Delete safely (use with care)	head -n 20 file	First 20 lines
grep -r "text" dir	Search text recursively	tail -f file	Follow log file
sed -i 's/a/b/g' file	Replace text in file	find / -name file	Find file
awk 'print \$1' file	Print first column	tar -czf file.tar.gz dir	Create archive
cut -d',' -f1 file	Cut first field (CSV)	tar -xzf file.tar.gz	Extract archive
sort file	Sort lines	zip -r file.zip dir	Create zip
uniq file	Remove duplicate lines	unzip file.zip	Extract zip
wc -l file	Count lines	curl -s url	Fetch URL silently
chmod 755 file	Change permissions	wget -q url	Download file
ps aux	List running processes	ping -c 4 host	Test connectivity
top	System performance	ss -tulpn	List open ports
df -h	Disk usage	netstat -tulpn	(legacy) open ports
du -sh *	Folder sizes	systemctl status svc	Service status
crontab -l	List cron jobs	journalctl -u svc -f	Follow service logs

③ COMMON BEGINNER MISTAKES

- ✗ Not quoting variables: \$var vs "\$var"
- ✗ Not checking exit codes
- ✗ Hardcoding paths and passwords
- ✗ Not using set -euo pipefail
- ✗ Overusing global variables
- ✗ Ignoring errors and log output
- ✗ Running scripts as root unnecessarily
- ✗ Not testing edge cases
- ✗ Not documenting scripts
- ✗ Deleting files without validation

TIP

Slow is smooth,
smooth is fast.

Write it right,
run it everywhere.

④ TROUBLESHOOTING CHECKLIST

- ✓ Script not running? → Check path, permissions, shebang
- ✓ Command not found? → Check PATH and install packages
- ✓ Permission denied? → Check ownership and chmod
- ✓ No output? → Check logs, redirects, variables
- ✓ Wrong output? → Check logic and input data
- ✓ Job not scheduled? → Check crontab and environment
- ✓ Performance issue? → Check loops, I/O, and resources
- ✓ Intermittent failure? → Check race conditions, retries

⑤ DEBUGGING WORKFLOW



Use small test data, reproduce often, and remove debug code after fixing.

⑦ SENIOR ENGINEER INSIGHTS

- ★ Great scripts are simple, predictable and boring.
- ★ Design for failure: expect things to break.
- ★ Logs are your best friend at 2 AM.
- ★ Idempotency saves you during incidents.
- ★ Security is not optional in production.
- ★ Automate everything, including your automations.

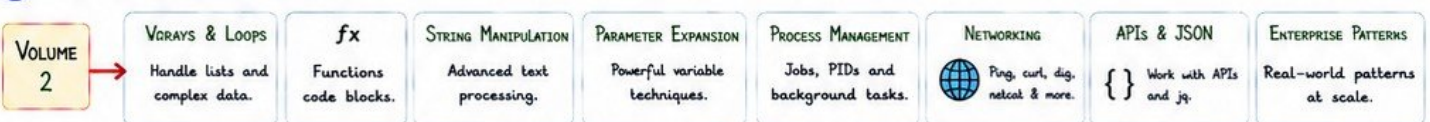
REMEMBER

The best scripts solve real problems, save time and reduce human errors.

⑧ KEY TAKEAWAYS



⑨ WHAT'S NEXT IN VOLUME 2



Instagram:
@veriqta



LinkedIn:
VERIQTA



X:
@veriqta



YouTube:
VERIQTA

VERIQTA

